Лабораторная работа: Kubernetes

Вита-Виктория Гиалтсо-Мижа

Группа j3313
ИСУ 466703

1. Среда и контрольный расчёт

На Mac с ARM64 поднята отдельная виртуальная машина Colima: 4 CPU, 4 GiB памяти и диск данных 12 GiB. В ней создан один узел kind. Кластер предназначен для учебных экспериментов, отказоустойчивость нескольких физических машин здесь не проверяется.

Параметр	Значение
Кластер / namespace	kind-vk-hw5 / vk-hw5
Узел	vk-hw5-control-plane
Kubernetes	v1.37.0
CPU capacity / allocatable	4 / 4
Память allocatable	3997988Ki
Расчётный образ	vk-pairs:hw5; Python 3.12-slim
Хранилище	RustFS Deployment + Service + PersistentVolumeClaim

Использован тот же файл vacancies.jsonl из ДЗ №3: 10 000 строк. Фильтр выбирает отдельное слово Go или Golang в названии вакансии. Навыки приводятся к нижнему регистру, пробелы по краям удаляются, повторы внутри вакансии исключаются. Каждая неупорядоченная пара учитывается один раз на вакансию.

```
Контроль ДЗ №3
Вакансий варианта: 124
Уникальных пар: 1008
Сумма частот пар: 2382
Полный словарь частот совпал с hw3_reference.json
```

Сначала Job читает исходный файл из локального образа. Затем этот же файл загружается в S3 внутри кластера. Четыре indexed-задачи читают разные части, а отдельный reducer суммирует частоты. Полный исходный файл не публикуется в репозитории.

2. Память: OOMKilled и рабочий предел

Запуск	Лимит памяти	Результат
pairs-baseline	256 MiB	Completed; совпадение с ДЗ №3
pairs-oom	8 MiB	OOMKilled, exit code 137
pairs-sized	32 MiB	Completed; совпадение с ДЗ №3

После подсчёта в исходном запуске memory.peak cgroup равен 17.39 MiB. Пик RSS процесса по getrusage - 24.88 MiB. Эти показатели не обязаны совпадать: RSS и учёт cgroup по-разному учитывают общие страницы и файловый кеш.

Рабочий предел выбран с запасом 50% от измерения cgroup и округлением вверх до 8 MiB: ceil(1.5 × 17.39 / 8) × 8 = 32 MiB. При этом он выше и измеренного RSS. Слишком маленький предел 8 MiB выбран для отрицательного опыта: он не позволяет завершить запуск Python и расчёт.

```
kubectl get pods -n vk-hw5 -l job-name=pairs-oom -o json
reason: OOMKilled
exitCode: 137
```

```
pairs-sized
rows: 124
unique_pairs: 1008
peak_rss_bytes: 26267648
cgroup_peak_bytes: 15204352
matches_hw3: true
```

memory limit ограничивает потребление памяти и при нехватке может привести к OOM kill. CPU limit обычно приводит к throttling, а не к OOMKilled. Request нужен планировщику для размещения; это отдельное свойство, не замена limit. Измерение относится к этому небольшому файлу и этому образу, для большего входа предел нужно измерять заново.

Доказательства: logs/pairs-oom-pods.txt, logs/pairs-sized-pods.txt, results/memory.json.

3. S3 и параллельный расчёт

RustFS работает в namespace vk-hw5, адрес для задач - http://rustfs:9000. Бакет hh-lake содержит исходный объект raw/vacancies.jsonl размером 3067413 байт. Данные лежат в PVC на локальном томе узла; пересоздание Pod не удаляет их, но удаление самого kind-кластера требует повторной загрузки.

Файл разделён по границам строк на четыре непересекающиеся части по 2500 строк. Job использует completionMode: Indexed, completions: 4 и parallelism: 4. Частота считается внутри каждой части; reducer складывает счётчики. Нормализация навыков и фильтр совпадают с контрольным расчётом.

Часть	Вакансий варианта	Уникальных пар	Сумма частот
0.json	34	621	878
1.json	43	508	729
2.json	20	194	258
3.json	27	388	517

Итог reducer: 124 вакансии, 1008 уникальных пар, 2382 вхождения. Из S3 повторно скачан результат, и все 1008 частот сравнивались с ДЗ №3. Совпадение только первых строк не использовалось как достаточная проверка. При равных частотах пары упорядочены лексикографически.

Пара навыков	Частота
golang + postgresql	38
go + golang	29
golang + kubernetes	23
golang + sql	23
docker + golang	22
docker + postgresql	21
postgresql + sql	19
git + golang	18
golang + linux	18
git + postgresql	17

Итоги: results/pairs.json, results/parallel.json, results/s3_objects.json; логи каждого mapper и reducer находятся в logs/. На файле 3 MB накладные расходы Kubernetes и S3 заметны, поэтому по этому опыту не заявляется ускорение в четыре раза.

4. Планировщик, приоритеты и квота

Ожидание CPU

Pod cpu-blocker запросил 2 CPU. После него создан cpu-waiting с таким же request. С учётом системных Pod и RustFS обе заявки не помещались на узле одновременно. Чтобы успеть наблюдать состояние, после подсчёта процесс удерживался sleep 300. Это опыт над резервированием ресурсов, а не тест фактической загрузки процессора.

```
cpu-waiting: Pending
reason: Unschedulable
0/1 nodes are available: 1 Insufficient cpu. preemption: 0/1 nodes are available: 1 No preemption victims found for incoming pod.
```

После удаления cpu-blocker ожидающий Pod стал Ready. Это показывает, что планировщик учитывает requests даже при низком текущем CPU usage.

Вытеснение

PriorityClass vk-hw5-low имеет значение 1000, vk-hw5-high - 2000. Сначала запущен priority-low с request 2 CPU, затем priority-high с тем же request. Высокоприоритетный Pod стал Ready, а для priority-low записано событие Preempted.

```
reason: Preempted
Preempted by pod ed67ae52-00a5-4e50-a901-a380e30ba2ec on node vk-hw5-control-plane
```

Квота namespace

В vk-hw5-quota установлена ResourceQuota: requests.cpu = 500m, requests.memory = 128Mi, limits.cpu = 1, limits.memory = 256Mi, pods = 2. Создание Pod с CPU request 600m отклонено на этапе API admission.

```
Forbidden: exceeded quota: compute-budget
requested: requests.cpu=600m
used: requests.cpu=0
limited: requests.cpu=500m
```

Отказ квоты отличается от ожидания CPU: при квоте Pod вообще не создан, а при нехватке ресурса узла объект Pod существует и находится в Pending. Проверки: logs/cpu-pending.txt, logs/cpu-after-release.txt, logs/priority-events.txt, logs/quota-rejection.txt.

5. YARN, Mesos и Kubernetes

Соответствия ниже описывают близкие роли. Компоненты не являются точными взаимозаменяемыми аналогами.

Роль	YARN	Mesos	Kubernetes
Ресурсы кластера	ResourceManager	Master / allocator	API server, scheduler и контроллеры
Работа на узле	NodeManager	Agent и executor	kubelet и container runtime
Координация приложения	ApplicationMaster	Framework scheduler	Job controller; для сложных приложений свой driver/operator
Размещение	RM выделяет контейнеры по заявкам AM	Framework принимает или отклоняет resource offer	kube-scheduler назначает Pod узлу
Единица ресурса	YARN container	Ресурсы offer и задачи executor	Pod с одним или несколькими контейнерами
Политика разделения	Очереди Capacity/Fair	Allocator, роли, квоты	Requests/limits, приоритеты, ResourceQuota
Отказоустойчивость управления	HA ResourceManager	Active/standby master, ZooKeeper	HA control plane, etcd и leader election

Почему один общий кластер

Выделенные кластеры MapReduce и Spark могут быть загружены неравномерно: один простаивает, пока другой перегружен. Общий менеджер позволяет перераспределять мощности между приложениями. При общем доступе к данным также не приходится держать отдельную копию набора для каждого фреймворка.

Централизованная и двухуровневая модель

В YARN глобальный ResourceManager принимает заявки ApplicationMaster и выделяет контейнеры. В Mesos первый уровень предлагает ресурсы, а scheduler конкретного фреймворка решает, подходят ли они его задачам. Это удобно для разных фреймворков с собственными требованиями, но требует согласования распределения между ними. В Kubernetes обычное размещение Pod централизовано в scheduler; вычислительную логику приложения он сам не знает.

Важное различие: YARN container означает выделенный набор ресурсов, а не обязательно Docker/OCI-контейнер. ApplicationMaster координирует конкретное приложение; обычный Kubernetes Job controller не реализует всю логику Spark или произвольный DAG.

6. Ответы по презентации

Три сущности YARN и запуск приложения

ResourceManager распределяет ресурсы всего кластера. NodeManager запускает и контролирует контейнеры на своём узле. ApplicationMaster принадлежит одному приложению: получает контейнеры, назначает им задачи и следит за выполнением. Сначала клиент запускает AM через RM, затем AM договаривается о дополнительных контейнерах и работает с NodeManager.

Как проходит resource offer в Mesos

Master сообщает scheduler фреймворка о доступных ресурсах. Scheduler принимает предложение и указывает задачи либо отклоняет его. При принятии master передаёт запуск agent, а задачи исполняет executor. Отклонённые ресурсы могут быть предложены другим подходящим фреймворкам: они не обязаны простаивать в ожидании нового предложения именно тому же scheduler.

Отказ master и ZooKeeper

Одновременно работает несколько master: один active и остальные standby. ZooKeeper участвует в выборе лидера и обнаружении активного master клиентами, agent и scheduler. После отказа выбирается новый active. Это не означает автоматического exactly-once выполнения любой пользовательской задачи: обработку повторов и состояние приложения тоже нужно учитывать.

Большая batch-задача и короткий запрос при FIFO

Если большая задача заняла все ресурсы, короткий запрос при FIFO будет ждать. Fair Scheduler позволяет разделять ресурсы между приложениями, а Capacity Scheduler - выделять гарантированные доли очередям. Если свободных контейнеров нет, одного изменения порядка очереди недостаточно для немедленного запуска: нужна настроенная preemption либо освобождение ресурсов.

Зачем несколько scheduler для разных фреймворков

У MapReduce, Spark и интерактивных сервисов разные требования к локальности, размеру групп задач и времени ожидания. Двухуровневая модель оставляет эти решения scheduler фреймворка, а общий уровень распределяет пул ресурсов. Это гибкость, но не гарантия лучшего результата на любом наборе задач.

Вопросы и определения сверены со слайдами 10, 12-17, 19-22 и 24-28 лекции К. Хицко «Менеджеры ресурсов YARN и Mesos».

7. Воспроизведение и файлы

Код и журналы публикуются в папке hw5-kubernetes репозитория Vitamj/data-engineering-hw3. Для повторения нужен тот же data/vacancies.jsonl из ДЗ №3. Полные данные и credentials исключены из Git; контрольные частоты находятся в results/hw3_reference.json.

```
docker build -t vk-pairs:hw5 .
kind create cluster --name vk-hw5 --kubeconfig runtime/kubeconfig
kind load docker-image vk-pairs:hw5 --name vk-hw5
python3.12 run_experiments.py
python3.12 collect_results.py
```

Перед запуском Docker должен работать. На этом Mac использовался профиль Colima vk-hw5. Если образы не загружаются напрямую из VM из-за TLS-сбоев, их можно скачать через crane на хосте, импортировать docker load и загрузить kind load docker-image. Имена namespace и кластера выделены только для этой работы.

Файлы	Назначение
Dockerfile, src/	Образ и программа расчёта
manifests/	Job, RustFS, том, приоритеты и квота
run_experiments.py	Последовательность экспериментов и запись журналов
collect_results.py	Повторное чтение S3 и проверка доказательств
logs/	Выводы команд, состояния Pod и события
results/	Полные частоты, измерения и сверка с ДЗ №3

Источники

Kubernetes: requests и limits
https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

Kubernetes: приоритеты и preemption
https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/

Kubernetes: ResourceQuota
https://kubernetes.io/docs/concepts/policy/resource-quotas/

Apache Hadoop: YARN
https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/YARN

Apache Mesos: Scheduler HTTP API
https://mesos.apache.org/documentation/scheduler-http-api/

Apache Mesos: High Availability
https://mesos.apache.org/documentation/high-availability/